

Criterio	Insuficiente (0-4)	Bien/Notable (5-8)	Sobresaliente (9-10)	RA/CE	Evaluación	Nota
Modelado UML y Documentación (RA5, RA6, RA4)						
Diagramas UML (Clases y Comportamiento)	No entrega diagramas o no coinciden con el código Java.	Entrega diagramas de clases y actividades/estados en formato imagen.	Mermaid: Diagramas integrados en Markdown. Reflejan ingeniería inversa y evolución del código.	RA5: b, d, f; RA6: d, f, h	Solo veo un commit en los diagramas, directamente haciendo ingeniería inversa. No veo los diagramas creados a priori. O no se entendió bien lo que se pedía o simplemente se hizo a posteriori. De haber defensa oral, habría que comprobar si se entienden los diagramas creados por la IA.	8
Documentación Técnica	No hay documentación o es escasa.	Documentación estática en PDF o Word detallando las clases.	GitHub Pages: Documentación en Markdown y Javadoc auto-generado y actualizado mediante CI/CD.	RA4: g; RA2: c	"documentación técnica detallada en cada clase y método público" está bastante exagerado: El workflow de github actions ha dado repetidos fallos. Probablemente, porque el workflow copia desde target/reports/apidocs/*, pero al ejecutar mvn javadoc:javadoc se genera en target/site/apidocs/. He comprobado localmente que target/reports/apidocs/index.html no existe y target/site/apidocs/index.html sí.	8,5
Control de Versiones y Flujo de Trabajo (RA4)						
Metodología GitFlow	Un solo commit o uso de una única rama (main).	Uso de ramas para separar funciones, pero sin estructura clara.	GitFlow Estricto: Uso de main, dev, features, release y hotfixes. Cierre de features tras integración.	RA4: f, h	Tiene hechas varias features, algunas probablemente sin identidad suficiente, pero así es como se pedía en el ejercicio. Main solo con commit inicial y merges por pull request. Viene una release 2.0.0 (not e lo tengo en cuenta pero ya veremos cómo va los de las versiones con más detenimiento).	10
Gestión de Repositorio	Repositorio desorganizado o inexistente.	Uso de repositorio remoto con commits regulares.	Protección de Main: Solo cambios vía PR desde develop. Reglas que exigen pasar el flujo de CI para mergear.	RA4: f, h, i	Veo pull requests, y no hay commits hay main. Pero creoq ue no hay protección como pido. Habría que comprobarlo en la defensa, si la hubiese.	10
Verificación, Pruebas y CI/CD (RA3, RA4)						
Pruebas y Calidad	No realiza pruebas unitarias de los métodos críticos.	Realiza pruebas unitarias (JUnit) y usa el depurador del IDE para corregir errores.	CI/CD con Actions: Ejecución automática de pruebas y reporte de cobertura en cada push a develop.	RA3: b, f, g; RA4: i	Sí, pero los últimos workflows han estado fallando.	9
Automatización	Procesos manuales de compilación y prueba.	Genera ejecutables desde el IDE y conoce las fases de desarrollo.	GitHub Actions: Automatización completa de tests, Javadoc y despliegue en GitHub Pages.	RA4: i; RA2: e	Lo mismo.	9
Calidad del Código y Profesionalización (RA1, RA4)						
Inglés Técnico		Uso de inglés en nombres de variables, métodos (ej. isMovementValid), comentarios y mensajes de commit.			Parte de la documentación del proyecto está en castellano.	/
Limpieza y Estructura		Penalización: Se restarán puntos por código spaghetti, lógicas confusas, faltas de ortografía o mala defensa del proyecto.			En la siguiente hoja se listan algunos problemas/malas prácticas	-
Fundamentos (RA1)		Se evalúa la comprensión de la relación del programa con la memoria y las herramientas del sistema.			Depende de defensa, si la hubiera.	/
					NOTA INICIAL	9

Documentación inflada: se habla de “interfaz premium”, “CI/CD”, “documentación viva”, “persistencia activa”, pero parte no está realmente integrada. Posible inidcio claro de uso de IA
Guardar partida no guarda: en MainFrame.java (line 232), handleSave() solo muestra un mensaje y deja un comentario: Logic to call GameDAO would go here.
Usuario/sesión incompleta: currentUserId = -1 existe pero no hay flujo real de login en la UI.
Generación de sudokus floja: SudokuGenerator.java (line 65) elimina números al azar, pero no garantiza solución única. Para un Sudoku esto es una debilidad funcional importante.
Seguridad mejorable: contraseñas con SHA-256 simple en UserDAO, sin salt ni BCrypt/Argon2. A nivel DAW1 puede valer como intento, pero defenderlo como “seguro” sería débil.
Credenciales/configuración hardcodeada: MySQL local con root y contraseña vacía en DatabaseConnection.java (line 26).
Código algo mezclado: MainFrame concentra creación visual, eventos, lógica de pistas, idioma, tema, guardado y ranking. No es spaghetti extremo, pero sí hay acoplamiento UI/lógica.
Ruido en tests/cobertura: los tests pasan, pero JaCoCo muestra trazas por incompatibilidad con la versión de Java usada (Unsupported class file major version 69). No rompe el build, pero queda feo.